

Pesquisa: Hooks essenciais do React Native

useState - Gerencia o estado local de um componente

useEffect - Executa código após renders e em resposta a mudanças

useRef - Guarda referências sem causar re-renderização

Questões orientadoras

Q1. O que é um hook? Para que serve?

Hooks são ferramentas do React que permitem usar recursos como memória (estado) e ações (efeitos) em componentes escritos como funções, sem precisar de classes. Eles funcionam como atalhos prontos para resolver tarefas comuns, por exemplo guardar dados como cliques de um botão ou executar algo quando a página carrega, deixando o código mais simples, organizado e fácil de entender

Q2. Para cada um dos três hooks, explique: o que ele faz, quando você o usaria e escreva um exemplo mínimo de código.

1. **useState**: O **useState** funciona como a memória do componente. Ele cria uma variável que, quando mudado, avisa ao React para atualizar a tela automaticamente. É usado sempre que algo na tela precisar mudar após uma ação do usuário (cliques, digitação, carregamentos). Ele te entrega duas coisas: o valor atual (para você mostrar na tela) e uma função de alteração (para você atualizar esse valor). Exemplo código:

```
import { useState } from 'react';

function MyComponent() {
  const [age, setAge] = useState(28);
  const [name, setName] = useState('Taylor');
  const [todos, setTodos] = useState(() => createTodos());
  // ...
}
```

2. O `useEffect` permite executar um bloco de código em momentos específicos do funcionamento de um componente: quando ele aparece na tela, quando é atualizado ou quando é removido. É como definir uma regra do tipo “sempre que algo acontecer, faça isso”. Ele é muito usado para buscar dados de uma API assim que a página carrega, manter informações sincronizadas (como atualizar o título da aba do navegador quando algum valor muda) e lidar com eventos do sistema, como iniciar um temporizador ou reagir a ações do usuário, como pressionar teclas.

```
import { useState, useEffect } from 'react';
import { createConnection } from './chat.js';

function ChatRoom({ roomId }) {
  const [serverUrl, setServerUrl] = useState('https://localhost:1234');

  useEffect(() => {
    const connection = createConnection(serverUrl, roomId);
    connection.connect();
    return () => {
      connection.disconnect();
    };
  }, [serverUrl, roomId]);
  // ...
}
```

3. `useRef`: O `useRef` é como uma “caixinha” onde você guarda um valor que não se perde quando o código é executado novamente. Esse valor fica dentro de uma propriedade chamada `.current`, e a principal diferença é que, quando ele muda, a tela não é atualizada automaticamente. Ele é útil para acessar diretamente elementos da tela, como colocar o foco em um campo de texto ou controlar um vídeo, e também para guardar informações que precisam continuar existindo entre as execuções do código, como o ID de um temporizador

```
import { useRef } from 'react';

function MyComponent() {
  const intervalRef = useRef(0);
  const inputRef = useRef(null);
  // ...
}
```

Q3. O `useEffect` recebe um segundo parâmetro chamado array de dependências.

O que acontece quando esse array está vazio `[]` ? E quando não é passado nenhum array?

R: Quando o `useEffect` tem um array vazio (`[]`), ele roda só uma vez, logo que o componente aparece na tela. Isso é útil para fazer coisas iniciais, como buscar dados ou iniciar um timer. Quando você não coloca o array, o `useEffect` roda toda vez que o componente atualiza. Ou seja, ele executa várias vezes, sempre que algo muda, e isso pode causar problemas se não tomar cuidado

Q4. Qual a diferença entre usar `useRef` e `useState` para guardar um valor? Em que situação do React Native o `useRef` é especialmente útil?

A diferença principal é que o `useState` é para o que aparece na tela, e `useRef` é para o que acontece por trás. Quando você muda um valor com `useState`, a tela atualiza. Já com `useRef`, o valor muda, mas a tela não muda, o que deixa o app mais rápido quando não precisa mostrar esse valor. No React Native, o `useRef` é útil para controlar coisas diretamente, como dar foco em um campo de texto, limpar um input ou mover uma lista. Também é muito usado em animações, porque ele permite mudar valores o tempo todo sem deixar o app lento

Q5. Quais são as duas regras dos hooks (Rules of Hooks)? Por que não podemos chamar um hook dentro de um `if` ?

As regras dos Hooks são simples: você deve usá-los sempre no topo do componente, antes de qualquer `return`, e nunca dentro de `if`, loops ou funções internas. Além disso, eles só podem ser usados dentro de componentes do React ou em Hooks criados por você, não em funções comuns. O motivo de não poder usar dentro de um `if` é que o React depende da ordem em que os Hooks são chamados. Ele não identifica os estados pelo nome, mas pela posição em que aparecem. Se um Hook ficar dentro de um `if` e, em algum momento, essa condição não for atendida, ele deixa de ser executado e bagunça a ordem dos próximos. Isso faz com que o React associe os valores errados às variáveis,

causando bugs difíceis de entender. Por isso, os Hooks precisam sempre ser chamados na mesma ordem, em todas as execuções do componente